

Start by fetching a data source by running

Then you can launch analyze jobs, run feature selection, call AutoML to generate features and more.

The schema from the data source.

slug property [¶](#)

```
slug: str
```

Sanitized name. Equivalent to Pulse 'internal name'.

subclasses class-attribute instance-attribute [¶](#)

```
subclasses: List[JavaWrapper] = []
```

analyze [¶](#)

```
analyze(default_breakdowns: bool = True, wait: bool = False, job_profile: Optional[JobProfile] = None, job_properties: Optional[Dict[str, str]] = None) -> bool
```

Analyze this datasource with Pulse.

By analyzing a data source, Pulse will launch a job to compute approximate histograms for each numerical feature.

The default analysis configuration will also include histogram breakdowns with **all categorical variables**. For each categorical variable in the data source, there will be a new histogram in each numerical feature.

data sources with many categorical fields

It is recommended to set `default_breakdowns=False` when there are many categorical fields. Otherwise, the amount of metadata that is generated might compromise the performance of Pulse.

The datasource analysis will be run with Pulse's default config for datasource analysis.

Parameters:

Name	Type	Description	Default
<code>default_breakdowns</code> ¶	bool	Whether the analysis should include breakdowns for all categorical fields or not. Default is <code>True</code> .	<code>True</code>
<code>wait</code> ¶	bool	Whether to wait for the datasource analysis to finish. The data analysis outcome will be logged. Default is <code>False</code> .	<code>False</code>
<code>job_profile</code> ¶	JobProfile	Pulse UI custom job profile.	<code>None</code>
<code>job_properties</code> ¶	dict	Additional spark properties, e.g. <code>{"spark.executor.memory": "1g"}</code> . By default, no additional spark properties are added.	<code>None</code>

Returns:

Type	Description
bool	Whether the analysis started or not.

create classmethod [¶](#)

```
create(app: App, name: str, path: str, file_format: str = 'csv', schema: Schema | None = None, **kwargs)
```

Creates a new data source in Pulse.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	The App where the datasource will be created.	<i>required</i>
<code>name</code> ¶	<code>str</code>	The name of the data source as it appears in the UI.	<i>required</i>
<code>path</code> ¶	<code>str</code>	The path of the file containing the dataset.	<i>required</i>
<code>file_format</code> ¶	<code>str</code>	The input file format. Options are <code>csv</code> and <code>parquet</code> . Defaults to <code>csv</code> .	<code>'csv'</code>
<code>schema</code> ¶	<code>Schema</code>	The Pulse Schema to use. Optional. If not specified, the Schema will be inferred by Pulse. Only used for csv files.	<code>None</code>
<code>kwargs</code> ¶	<code>Dict</code>	Keyword arguments used for the configuration of the DataSource and for the SchemaInference job.	<code>{}</code>

Other Parameters:

Name	Type	Description
<code>has_header</code>	<code>bool</code>	Flags whether the dataset is to be read with header. Only used for csv files. Defaults to True.
<code>sep</code>	<code>str</code>	Delimiter to use. Should contain exactly one character. Only used for csv files. Defaults to <code>','</code> .
<code>missing_values</code>	<code>set of str</code>	Set of strings with values for empty value. Only used for csv files. Defaults to <code>{',', 'NA', 'NULL'}</code> .
<code>quote</code>	<code>str</code>	The character used to denote the start and end of a quoted item. Only used for csv files. Defaults to <code>"'</code> .
<code>line_separator</code>	<code>str</code>	The string used mark the end of line. Only used for csv files. Defaults to <code>'\n'</code> .
<code>encoding</code>	<code>str</code>	Encoding to use for UTF when reading/writing. Only used for csv files. Defaults to <code>'UTF-8'</code> .
<code>cat_threshold</code>	<code>int</code>	A value that represents the maximum number of distinct values on a given field for it to be inferred as categorical. Defaults to 15.
<code>job_properties</code>	<code>Dict</code>	The dictionary with the specific properties for the schema inference job. Will only be used if no Schema is specified. Defaults to Pulse PropertiesProfile.
<code>n_records</code>	<code>int</code>	The number of records that will be read per partition when performing datasource schema inference. Will only be used if no Schema is specified. Defaults to 15000.
<code>dtypes</code>	<code>Dict</code>	The dictionary with the field overriding information when inferring the datasource schema. Will only be used if no Schema is specified. Defaults to None.

Name	Type	Description
<code>merge_schema</code>	bool	Specify if schema merging should be allowed. Only used for parquet files. Defaults to False. See: https://spark.apache.org/docs/latest/sql-data-sources-parquet.html#schema-merging

Returns:

Type	Description
DataSource	DataSource object corresponding to the Pulse datasource created.

create_automl_fe_plan [↗](#)

```
create_automl_fe_plan(
    semantics: Sequence[SemanticField],
    timestamp_field: str,
    realtime_windows: Optional[Sequence[TimeModel]] = None,
    scheduled_windows: Optional[Sequence[TimeModel]] = None,
    plan_name: Optional[str] = None,
    profile_filter: Optional[str] = None,
    compute_unfiltered_metrics: bool = False
) -> JavaWrapper
```

Creates automl's feature engineering plan.

Parameters:

Name	Type	Description	Default
<code>semantics</code> ↗	sequence of SemanticField	Sequence of semantic objects to define the transformations.	<i>required</i>
<code>timestamp_field</code> ↗	str	The name of the timestamp field.	<i>required</i>
<code>realtime_windows</code> ↗	sequence of TimeModel	Realtime windows to use in the metrics.	None
<code>scheduled_windows</code> ↗	sequence of TimeModel	Scheduled windows to use in the metrics.	None
<code>plan_name</code> ↗	str	Name of the resulting feature engineering plan. In case it is not provided, a name is automatically generated.	None
<code>profile_filter</code> ↗	str	Filter to be used in metrics. Fields must be enclosed with the pattern <code>\${field}</code> .	None
<code>compute_unfiltered_metrics</code> ↗	bool	Boolean that controls if unfiltered metrics are also computed, in case a profile filter is defined. Default is False.	False

Returns:

Type	Description
JavaWrapper	The created plan.

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> dataset: DataSource = app.pullDataSourceByDesc("data")
>>> real_time_windows = (TimeModel(1, TimeUnit.DAY),)
>>> semantics = (
...     TaggedField("card", tags=["card", "grouping-entity"]),
...     TaggedField("ip", tags=["ip", "entity", "grouping-entity"]),
...     TaggedField("amount", tags=["amount"]),
...     CompositeField(
...         "card, customer", tags=["grouping-entity"], fields=["card", "customer"]
...     ),
... )
>>> plan = dataset.create_automl_fe_plan(
...     semantics=semantics,
...     timestamp_field="timestamp1",
...     realtime_windows=real_time_windows,
...     plan_name="test fe plan",
...     profile_filter="${event_type} == 'TRANSACTION' and ${scorable} == true"
... )
```

get_dtypes [¶](#)

```
get_dtypes(pyspark_context=False)
```

Returns a dictionary mapping field names to DataScienceFields.

Parameters:

Name	Type	Description	Default
<code>pyspark_context</code> ¶		Indicates if this is a pyspark context or a Pulse context. In a pyspark context the type of time fields will be given as timestamp in seconds (this is the default unit after the pulse_to_pyspark transformation), otherwise the actual type from Pulse will be returned.	False

Returns:

Type	Description
Dict	Dictionary with field description as keys and the corresponding DataScienceField type as values.

run_automl_feature_engineering [¶](#)

```
run_automl_feature_engineering(semantics: Sequence, timestamp_field: str = 'event_occurred_at', realtime_windows: Optional[Sequence[TimeModel]] = None, scheduled_windows: Optional[Sequence[TimeModel]] = None, plan_name: Optional[str] = None, output_datasource_name: Optional[str] = None, profile_filter: Optional[str] = None, compute_unfiltered_metrics: bool = False, output_path: Optional[str] = None) -> str
```

Runs automl's feature engineering on this datasource.

Parameters:

Name	Type	Description	Default
<code>semantics</code> ¶	sequence of SemanticField	Sequence of semantic objects to define the transformations.	required
<code>timestamp_field</code> ¶	str	The name of the timestamp field.	'event_occurred_at'

Name	Type	Description	Default
<code>realtime_windows</code> ↗	sequence of TimeModel	Realtime windows to use in the metrics.	None
<code>scheduled_windows</code> ↗	sequence of TimeModel	Scheduled windows to use in the metrics.	None
<code>plan_name</code> ↗	str	Name of the resulting feature engineering plan. In case it is not provided, a name is automatically generated.	None
<code>output_datasource_name</code> ↗	str	Name of the feature engineered datasource. In case it is not provided, the process created a derived name from the input source.	None
<code>profile_filter</code> ↗	str	Filter to be used in metrics. Fields must be enclosed with the pattern <code>\${field}</code> .	None
<code>compute_unfiltered_metrics</code> ↗	bool	Boolean that controls if unfiltered metrics are also computed, in case a profile filter is defined. Default is False.	False
<code>output_path</code> ↗	str	The path where the new datasource should be stored. In case it is not provided, the new datasource is saved in the source's path.	None

Returns:

Type	Description
str	The plan execution id.

Examples:

```
>>> app = Feedzai().app(name="sample")
>>> dataset: DataSource = app.pullDataSourceByDesc("data")
>>> real_time_windows = (TimeModel(1, TimeUnit.DAY),)
>>> semantics = (
...     TaggedField("card", tags=["card", "grouping-entity"]),
...     TaggedField("ip", tags=["ip", "entity", "grouping-entity"]),
...     TaggedField("amount", tags=["amount"]),
...     CompositeField(
...         "card, customer",
...         tags=["grouping-entity"], fields=["card", "customer"]
...     ),
... )
>>> plan_execution_id = dataset.run_automl_feature_engineering(
...     semantics=semantics,
...     timestamp_field="timestamp1",
...     realtime_windows=real_time_windows,
...     plan_name="test fe plan",
...     output_datasource_name="output data",
...     profile_filter="${event_type} == 'TRANSACTION' and ${scorable} == true"
... )
```

run_feature_selection [¶](#)

```
run_feature_selection(target_field: str, threshold: float = 0.9, sample_size: int = 200000, max_features: int | None = None, job_properties: dict[str, str] | None = None, job_profile: JobProfile | None = None)
```

Run feature selection on the Datasource.

Features are selected so that the cumulative sum of the feature importance on the selected features is below the given threshold. A maximum of max_features will be returned. The feature importance is computed from an H2O GBT model run on a sample of the data.

Parameters:

Name	Type	Description	Default
<code>target_field</code> ¶	str	The field to be used as target.	<i>required</i>
<code>threshold</code> ¶	float	Threshold on the cumulative feature importance, defaults to 0.9.	0.9
<code>sample_size</code> ¶	int	Sample size, defaults to 200,000.	200000
<code>max_features</code> ¶	int	Maximum number of features to return, defaults to Inf.	None
<code>job_properties</code> ¶	Mapping of str	Map of spark configurations for the job. If not provided defaults to the config of the current spark session.	None
<code>job_profile</code> ¶	JobProfile	Pulse UI custom job profile.	None

Notes

If both job_properties and job_profile are specified and there is intersection between their properties, the job_properties property will overwrite the job_profile property.

Returns:

Type	Description
set of str	The selected features.

Examples:

```
>>> selected_features = datasource.run_feature_selection(  
...     target_field="target"  
... )  
>>> job_profile = app.pullJobProfileByDesc("job_profile_name")  
>>> selected_features = datasource.run_feature_selection(  
...     target_field="target",  
...     threshold=0.7,  
...     sample_size=1_000_000,  
...     max_features=20,  
...     job_properties={  
...         "spark.master": "yarn",
```

```

...     "spark.app.name": "Feature Selection",
...     "spark.executor.memory": "1g",
...     "spark.dynamicAllocation.enabled": "True",
...     "spark.dynamicAllocation.maxExecutors": "4",
...     }
... )
>>> selected_features = datasource.run_feature_selection(
...     target_field="target",
...     threshold=0.7,
...     sample_size=1_000_000,
...     max_features=20,
...     job_profile=job_profile
... )

```

split [¶](#)

```

split(plan_name: str, splits: Set[Split], timestamp_field: str = 'event_occurred_at', scorable_filter: Optional[str] = None, wait: bool = False) -> Set[str]

```

Splits the datasource according to the splits passed.

Parameters:

Name	Type	Description	Default
<code>plan_name</code> ¶	<code>str</code>	Name that the resulting datasource split plan will have.	<i>required</i>
<code>splits</code> ¶	<code>set of Split</code>	A set of <code>Split</code> that will determine how the data source will be split. Must have at least one element.	<i>required</i>
<code>timestamp_field</code> ¶	<code>str</code>	The name of the timestamp field.	<code>'event_occurred_at'</code>
<code>scorable_filter</code> ¶	<code>str</code>	Optional string to filter scorable events. Fields must be enclosed with the pattern <code>\${field}</code> .	<code>None</code>
<code>wait</code> ¶	<code>bool</code>	Optional flag to indicate whether the method should wait for the datasource split to end before returning the designated method output.	<code>False</code>

Returns:

Type	Description
<code>set of str</code>	The names of the new datasources that the splitting process will create.

Examples:

```

>>> scorable = "(${amount} > 0)"
>>> splits = {
...     Split("train", 1350776834512, 1351173363000),
...     Split("test", 1351173363000, 1351473463000)
... }
>>> datasources = datasource.split(
...     plan_name="plan",
...     splits=splits,
...     timestamp_field="timestamp",

```



```
...     scorable_filter=scorable
... )
```

to_pyspark [↗](#)

```
to_pyspark(spark\_session: _TSparkSession) -> _TDataFrame
```

Convert datasource to pyspark.

Parameters:

Name	Type	Description	Default
spark_session ↗	<code>pyspark.sql.SparkSession</code> .	Spark session to use. Defaults to Pulse's session.	<i>required</i>

Returns:

Type	Description
<code>DataFrame</code>	The converted pyspark dataframe.